Pergamon

Computers & Geosciences Vol. 24, No. 1, pp. 95–100, 1998

Printed in Great Britain
PII: S0098-3004(97)00137-4
0098-3004/98 $19.00 + 0.00

SHORT NOTE

A PRACTICAL IMPLEMENTATION OF THE BOX COUNTING ALGORITHM

GUIDO GONZATO

Dipartimento di Fisica, Settore di Geofisica, Università di Bologna, Viale Berti Pichat 8, 40100 Bologna, Italy
(*e-mail:* guido@ibogfs.df.unibo.it)

(Received 22 November 1996; revised 1 August 1997)

INTRODUCTION: FRACTALS AND ALGORITHMS

Fractal geometry and fractal analysis have had a huge impact on geosciences as a whole (see Scholz and Mandelbrot, 1989; Turcotte, 1992, and references therein), though most published works deal with fractals from a theoretical point of view. Branches of geosciences like geomorphology or geography, which could benefit from fractal analysis carried out on actual morphologic features, have in practice gained less than expected from the fractal approach.

This is perhaps a paradox, considering that the first instance of fractal analysis dates back to the works of a geographer, Richardson (1961), in his attempts to relate the length of a coastline and the length of the "ruler" used to measure it. The method he followed consisted in measuring the length of the coastline with a map divider, using increasingly shorter steps. Richardson found that

$$L(\eta_i) \propto \eta_i^{(1-D)} \tag{1}$$

where $L(\eta_i)$ is the length of the coastline as a function of η_i, η_i is the i-th spread of the map divider, and D is a constant. Mandelbrot (1967) interpreted D from a geometrical point of view, noting that Equation (1) coincided with the mathematical definition of Hausdorff–Besicovich dimension. In this respect, D is a dimension value; Mandelbrot called it "fractal dimension". An introduction to fractals and the techniques employed for estimating D can be found in Roach and Fowler (1993).

The procedure followed by Richardson is known as the "divider method" or "walker's ruler" (Mandelbrot, 1982) (see Fig. 1), and in practice it is never used because of its main limitation: the object being measured must be connected. Another algorithm is the box counting (cf. Grassberger, 1993) (see Fig. 2), which has no restrictions on the shape of the object. To calculate D, one covers the object with a grid of squares initially of side η_1, then counts the number N_1 of squares that include part of the object. This step is repeated M times, using squares of increasingly shorter side. The relation between η_i and D is

$$\log(N(\eta_i)) = a - D \log(\eta_i). \tag{2}$$

One then calculates D by fitting the regression line between the independent variable $\log(\eta_i)$ and the dependent variable $\log(N(\eta_i))$, where $i = 1, \ldots, M$. D is given by the absolute value of the line slope.

To illustrate practically the use of the method, let us imagine one wants to measure the fractal dimension of the fault lines distribution in an area: the first step is to draw a tectonic map. The following algorithm in Table 1 is then applied.

It must be stressed that in order to obtain reliable results and a realistic estimate of D, two conditions are to be met: (1) a large range of square side lengths must be used and (2) for each step, a large number of boxes must be "full" (that is, containing part of the object). Besides, particular care must be taken with the subsequent regression analysis, see Davis (1973), Draper and Smith (1981) and Koch (1987) for examples of application of this technique.

Manual implementation of the method is tedious and time consuming. As a result, the computer-shy scholar is bound either to give up or to employ only few square lengths, thus breaking condition 1. However, even a PC-class computer can generate η_i, $N(\eta_i)$ and D in a matter of few seconds. The following program has been written with ease of use and efficiency in mind.

THE PROGRAM `vsbc`

`vsbc` (virtual screen box counting) is a simple yet powerful implementation of the classical box-counting algorithm, aimed at fractal analysis of digitised maps. As its name suggests, it uses a "virtual screen" that is necessary to improve scope and performance.

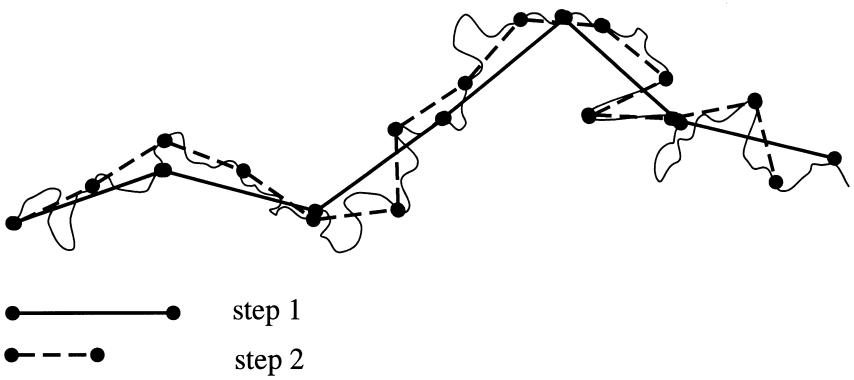

Figure 1. Divider method algorithm.

Table 1. Box counting algorithm

```
VAR LogEta, LogNBox:                                    ARRAY [1...30] of Real;
  ScreenSize, Side, x, y,
  NumSteps, NumBox:                                          Integer;
  D:                                                          Real;
BEGIN
  ScreenSize:= ''screen width'';                          { e.g. 1000 pixels }
  Side:= ScreenSize;                                      { initial box side }
  NumSteps:= 0;
  REPEAT
    NumBox:= 0;
    y:= 0;
      WHILE (y <=ScreenSize) DO BEGIN
        WHILE (x <=ScreenSize) DO BEGIN
          IF "the box (x,y,x+side,y+side) contains at least one pixel turned
          on" THEN
          NumBox:= NumBox+1;
        x:= x+Side
        END; { WHILE x }
        y:= y+Side
      END; { WHILE y }
      NumSteps:= NumSteps+1;
      LogEta(NumSteps):= log(Side);
      LogNBox(NumSteps):= log(NumBox);
      Side:= Side/sqrt(2)
    UNTIL (Side <=1);
    "calculate regression line between vectors LogEta and LogNBox, NumSteps
    data each";
    D:= abs("line slope")
END. {  of box counting algorithm }
```

The box-counting algorithm requires that the range of the box side length be as large as possible. As a rule of thumb, the range should span at least three orders of magnitude, which requires a 1000×1000-pixel screen. However, a standard PC computer screen has a resolution of 800×600 or 1024×768 pixels. Since the boxes must be square, this provides a usable 600×600 or 768×768 screen, which is hardly enough for an accurate analysis.

To extend the screen resolution further there are three ways. The first is to buy expensive hardware (namely, a big monitor and a memory-filled video card). The second is to employ the "virtual screen" facilities provided by the scrolling windows in some environments, such as MS Windows or X-Window System. The third way, which is the most effective and easiest to implement, is to drop the graphical

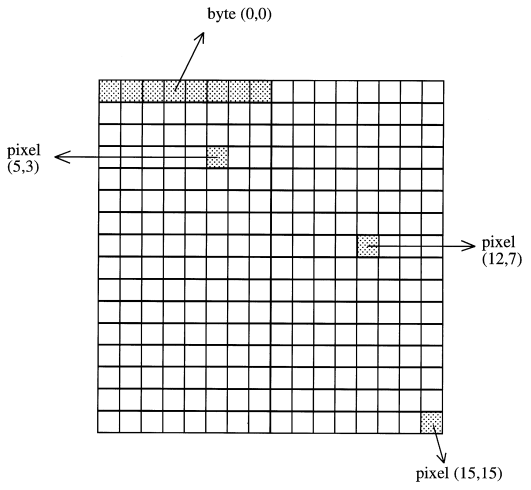

Figure 3. 2×16 array of bytes acting as 16×16-pixel virtual screen.

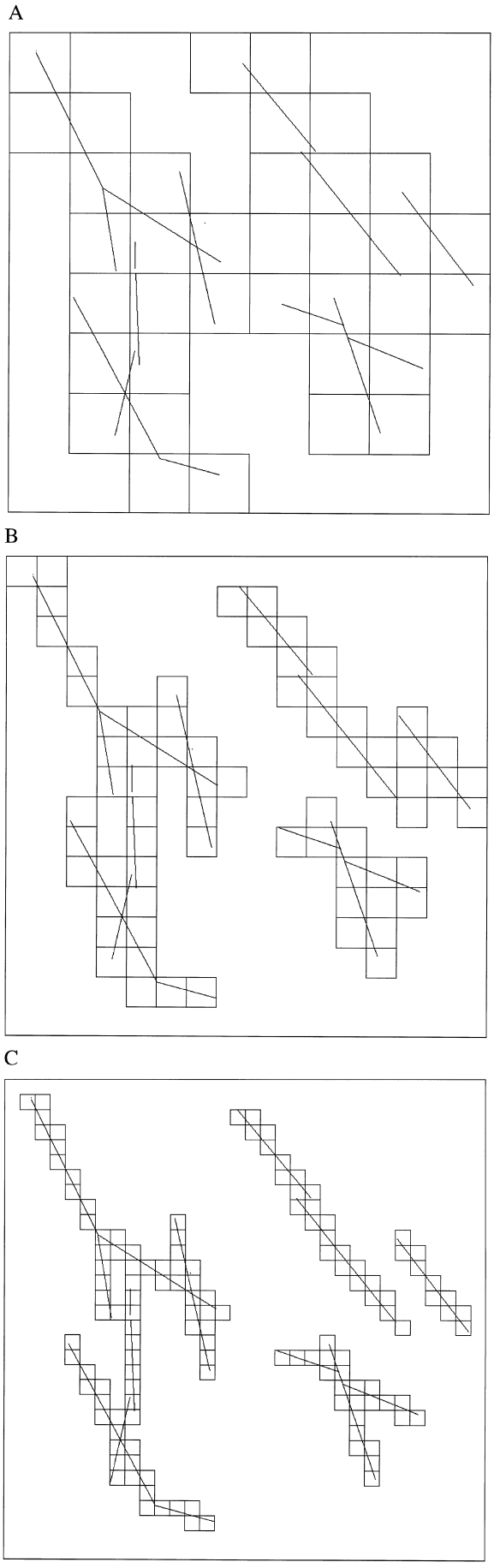

Figure 2. Box counting algorithm.

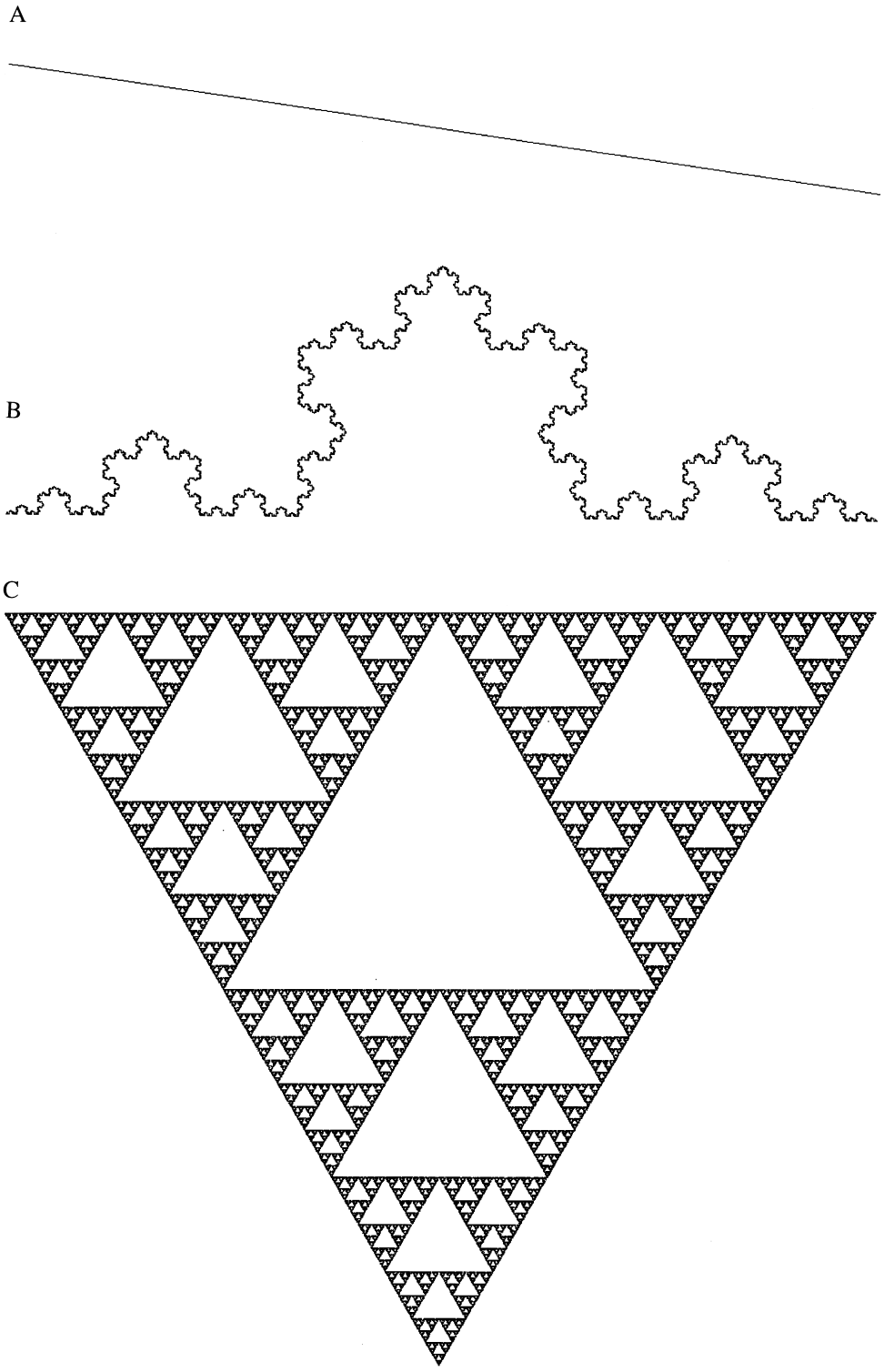

Figure 4. (A) straight line ($D = 1$). (B) Koch curve ($D = 1.2618$). (C) Sierpinsky carpet ($D = 1.5849$).

Table 2. Application of vsbc: theoretical and calculated D

Picture	Theoretical D	Calculated D	Std. Dev. (D)	Difference (%)
Straight line	1.0000	0.992	0.007	−0.8
Koch curve	1.2618	1.279	0.010	+1.36
Sierpinsky carpet	1.5849	1.5647	0.016	−1.27

approach completely and simulate a large screen using conventional memory.

An S*S-pixel wide monochromatic screen can be implemented using an $S/8$*S array of bytes, where each bit acts as a "virtual pixel" that is either on or off (Fig. 3). This implementation is simple, memory efficient, and very fast. In fact, accessing the single "pixels" in the virtual screen does not require any BIOS or graphic library function calls and their inherent overhead. Furthermore, the screen resolution can be increased at will to increase precision, with the only limit being available memory.

Other practical details that can influence the precision of the measured D are the scanner resolution, the displacement and the angle of the image on the virtual screen. According to our preliminary tests, the former point does not seem to affect D significantly, whereas the latter is likely to be the main cause of error, and will be the subject of a subsequent article.

IMPLEMENTATION

vsbc is the translation into ANSI C of the algorithm in Table 1. The program consists of two modules: the first, vscrn.c, contains the routines for the virtual screen management, the second is the main program.

The module vscrn.c implements all the basic functions needed for graphics. In particular, the functions putpixel(), vgetpixel() and vline() are used to plot pixels, to read the pixels status (on or off), and to draw lines; the functions vpcxsave() and vpcxload() are used to save or load a virtual screen as a monochromatic PCX file. This graphic format is particularly suitable for black and white pictures, as it is simple to implement and produces very compact files that are amenable to further compression, saving disk space. Its only drawback is that it is becoming obsolete and being replaced by other formats, like GIF or TIFF; as a result, not all graphic packages still support it. If necessary, vpcxsave() and vpcxload() can be replaced by other functions implementing, say, the TIFF format.

As it stands, vsbc uses a 2048×2048 screen, therefore using only 2048*2048/8 = 524,288 bytes for its array. This small amount of memory consumption makes it possible to use the program even on PC-class machines; to increase the screen width, one changes the directive #define NPIXELS 2048 in vscrn.c.

The critical step in the box counting algorithm is when a box is explored to check whether it contains data. In vsbc the search is simple, with the boxes systematically searched, pixel by pixel, until either at least a full pixel is encountered or the last empty pixel is reached. Fast search techniques are beyond the scope of this work; see Grassberger (1993) or Wirth (1976) for further insight.

vsbc has been compiled and tested using the C compiler gcc v. 2.7.0 under Linux 1.2.13, as well as cc under Digital Unix 4.0, Djgpp v. 2.01 and Turbo C++ 3.0 under MS-DOS. The program is available by anonymous FTP on ibogeo.df.unibo.it (137.204.48.112), in directory /pub/vsbc, or from FTP.IAMG.ORG.

RESULTS

In order to test vsbc, a few different images have been generated, saved as PCX files, and submitted to the program (Fig. 4a–c). These pictures represent a straight line, the Koch curve, and the Sierpinsky carpet; their theoretical dimension values are 1, 1.2618 and 1.5849, respectively.

For each picture, vsbc outputs a file containing pairs of values log(box side)–log(number of boxes), on which one calculates the regression line, and finally D. The results are shown in Table 2. Note how the calculated D is close to the theoretical value.

CONCLUSIONS

In our experience, vsbc has proven to be a practical and useful program. We hope that its use may help the computer-shy researcher develop further insight into the application of fractal geometry on real-world objects.

Acknowledgments—This work was performed with contributions from CNR-Gruppo Nazionale per la Vulcanologia and Gruppo Nazionale Difesa dai Terremot.

REFERENCES

Davis, J. C. (1973) *Statistics and Data Analysis in Geology*. John Wiley & Sons, New York, 646 pp.

Draper, N. and Smith, H. (1981) *Applied Regression Analysis*, 2nd edn. John Wiley & Sons, New York, 709 pp.

Grassberger, P. (1993) On efficient box counting algorithms. *International Journal of Modern Physics C* **4**(3), 515–523.

Koch, K. R. (1987) *Parameter Estimation and Hypothesis Testing in Linear Models*. Springer Verlag, Berlin, 378 pp.

Mandelbrot, B. B. (1967) How long is the coast of Britain? Statistical self-similarity and fractional dimension *Science* **156**, 636–638.

Mandelbrot, B. B. (1982) *The Fractal Geometry of Nature*. W. H. Freeman and Co., San Francisco, California, 468 pp.

Richardson, L. F. (1961) The problem of contiguity: an appendix of statistics of deadly quarrels. *General Systems Yearbook* **6**, 139–187.

Roach, D. E. and Fowler, A. D. (1993) Dimensionality analysis of patterns: fractal measurements. *Computers & Geosciences* **19**(6), 849–869.

Scholz, H. and Mandelbrot, B. B. (eds.) (1989) *Fractals in Geophysics*. Birkhäuser Verlag, Basel, 313 pp.

Turcotte, D. L. (1992) *Fractals and Chaos in Geology and Geophysics*. Cambridge University Press, Cambridge, 221 pp.

Wirth, N. (1976) *Algorithms + Data Structures = Programs*. Prentice-Hall, Englewood Cliffs, New Jersey, 228 pp.